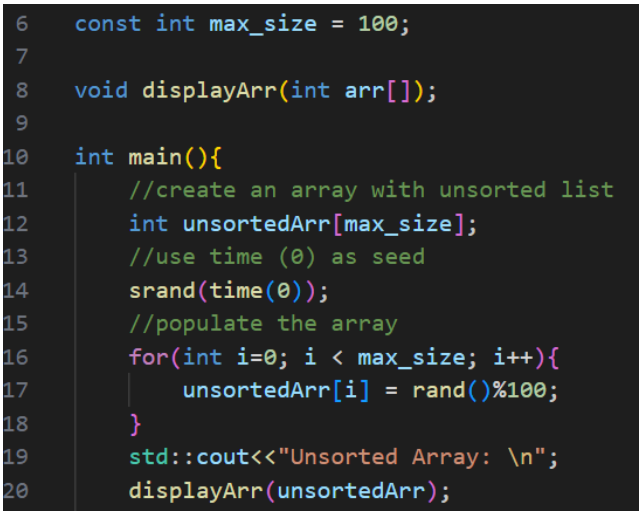
T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 8, 2026
NAME:	Macasaet, John David	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA NO. 7.1 Sorting Algorithms Pt 1

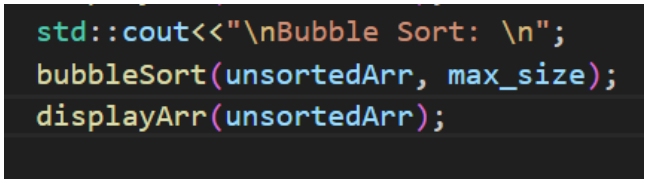
1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	**To generate an array with random unsorted elements, we used srand() and rand(). And using a for loop we populated the array with the random 100 numbers.**

```
6   const int max_size = 100;
7
8   void displayArr(int arr[]);
9
10  int main(){
11      //create an array with unsorted list
12      int unsortedArr[max_size];
13      //use time (0) as seed
14      srand(time(0));
15      //populate the array
16      for(int i=0; i < max_size; i++){
17          unsortedArr[i] = rand()%100;
18      }
19      std::cout<<"Unsorted Array: \n";
20      displayArr(unsortedArr);
```

```
cd "c:\Users\LENOVO\Downloads\HOA 7.1\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Unsorted Array:
94 86 7 71 50 42 3 64 26 66 37 94 0 87 75 82 20 13 93 30 57 4 81 67 51 85 76 3 98 19 32 18 94 18 59 46 61 75 38 76 23 74 17
 96 28 9 42 60 75 90 80 57 5 14 62 55 11 19 80 99 96 47 73 41 46 37 18 50 48 45 10 42 21 75 18 66 7 1 43 42 65 18 50 37 21
93 43 21 64 57 33 53 1 88 5 76 41 95 57 64
PS C:\Users\LENOVO\Downloads\HOA 7.1>
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
std::cout<<"\nBubble Sort: \n";
bubbleSort(unsortedArr, max_size);
displayArr(unsortedArr);
```

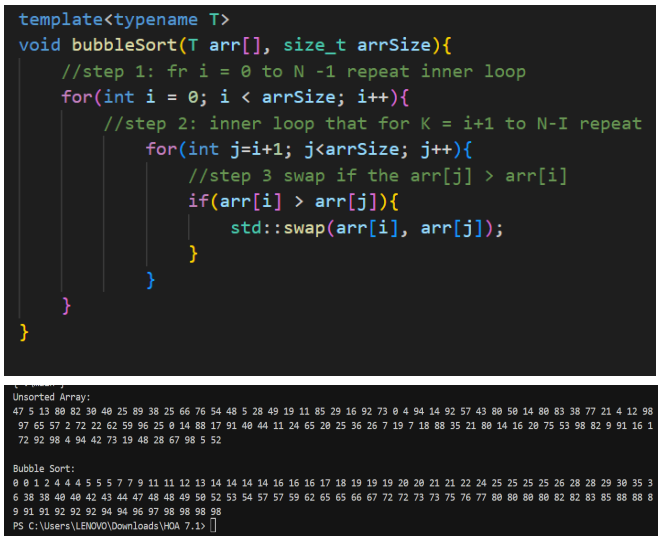
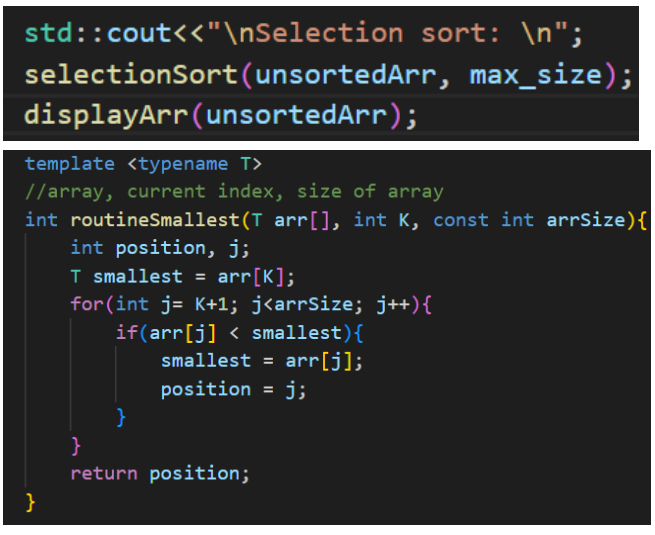
	
Observations	**Bubble Sort has a worst case time complexity of O(n^2) and a best case of O(n). This is because there is one outer for loop and an inner for loop thus creating a nested for loop. Bubble sort works by having the program look at the next number and seeing if the current number is bigger or smaller than the next number then switching places if the current is bigger.**

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

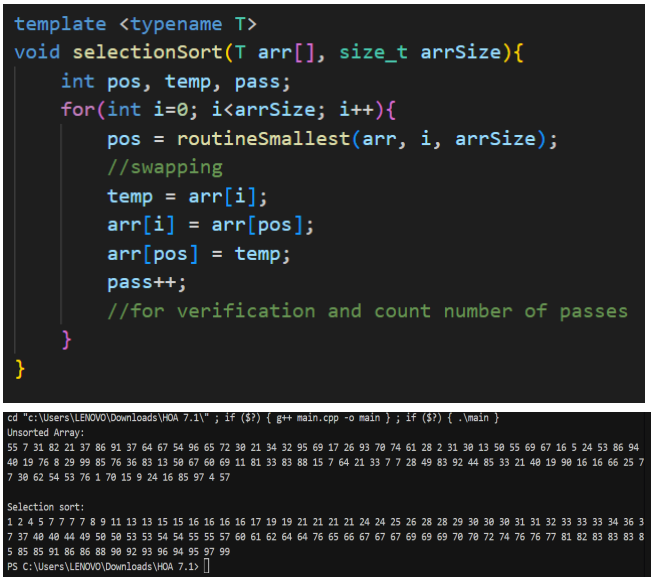
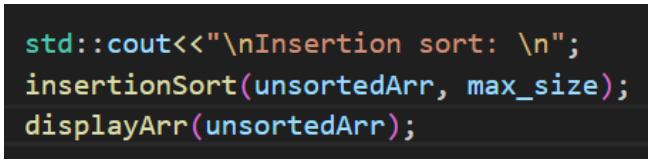
	
Observations	**Selection Sort has a worst and best case time complexity of O(n^2) because it has 1 for loop in the main selection function and there is also another for loop inside routineSmallest. Selection Sort works by finding the smallest number and putting it in the front. This continues until the array gets sorted.**

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	

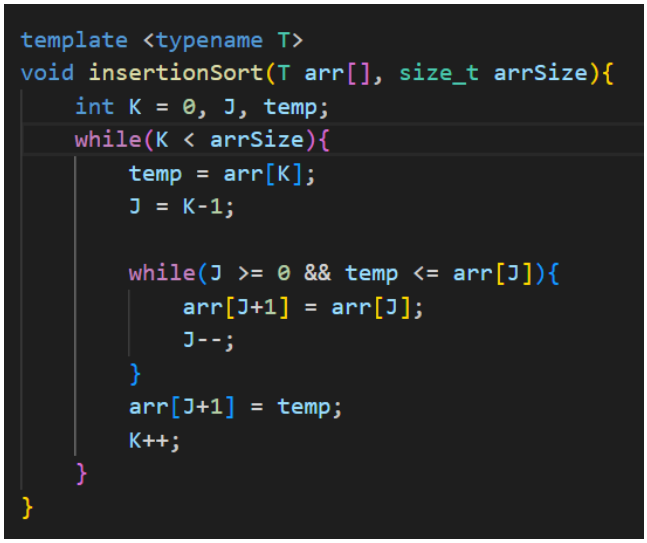
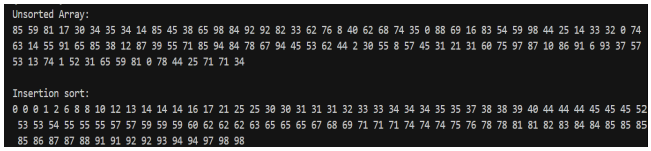
	 
Observations	**Insertion sort has a worst case time complexity of O(n^2) and a best case of O(n). The worst case is not logarithmic because the while loop does not divide by half. Insertion sort works by taking from an unsorted side and then putting in on the sorted side.**

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE:

CPP File	Header File (insertionSort)	Header File (linearSearchArr)
START FOR (i=0; i < 100; i++) unsortedArr[] = random numbers valued 1-5 DISPLAY unsortedArr SORT unsortedArr using Insertion Sort DISPLAY sorted array DISPLAY Candidate 1 linearSearchArr(100, 1, sortedArr) DISPLAY Candidate 2 linearSearchArr(100, 2,	insertionSort(arr, size) int k = 0, J, temp WHILE (k < arrSize) temp = arr[k] J = k-1 WHILE(J>=0 and temp<= arr[J]) arr[J+1] = arr[J] J-- arr[J+1] = temp k++	linearSearchArr(size, item, arr) int count = 0 FOR(i=0;i<100;i++) IF(arr[i] == item) count++ RETURN count

sortedArr) DISPLAY Candidate 3 linearSearchArr(100, 3, sortedArr) DISPLAY Candidate 4 linearSearchArr(100, 4, sortedArr) DISPLAY Candidate 5 linearSearchArr(100, 5, sortedArr) FOR (i=0; i<5; i++) voteTally[i] = linearSearchArr(100, i+1, sorted array) tempArr[i] = voteTally[i] SORT voteTally[] FOR(i=0; i< 5; i++) IF(tempArr[i] == tallyVotes[4]) DISPLAY Winner Candidate i END		

Screenshot of Code	supplementary.cpp:
	`#include<iostream>`
	`#include<cstdlib>`
	`#include "sortingAlgo.h"`
	`#include "searchingAlgo.h"`
	
	`const int max_size = 100;`
	
	`void displayArr(int arr[]);`
	
	`int main(){`
	`//create an array with unsorted list`
	`int unsortedArr[max_size];`
	`//use time (0) as seed`
	`srand(time(0));`
	`//populate the array`
	`for(int i=0; i < max_size; i++){`
	`unsortedArr[i] = (rand() % 5) + 1;`
	`}`
	`std::cout<<"Unsorted Votes: \n";`
	`displayArr(unsortedArr);`
	
	`std::cout<<"\nSorted using Insertion Sort: \n";`
	`insertionSort(unsortedArr, max_size);`
	`displayArr(unsortedArr);`

```
    std::cout<<"\nTally of Votes: ";
    std::cout<<"\nCandidate 1: Bo Dalton Capistrano: "<<linearSearchArr(max_size, 1, unsortedArr);
    std::cout<<"\nCandidate 2: Cornelius Raymon Agustin: "<<linearSearchArr(max_size, 2, unsortedArr);
    std::cout<<"\nCandidate 3: Deja Jayla Banaga: "<<linearSearchArr(max_size, 3, unsortedArr);
    std::cout<<"\nCandidate 4: Lalla Brielle Yabut: "<<linearSearchArr(max_size, 4, unsortedArr);
    std::cout<<"\nCandidate 5: Franklin Relano Castro: "<<linearSearchArr(max_size, 5, unsortedArr);

    int totalVotes[5];
    int tempArr[5];
    for(int i=0; i < 5; i++){
        totalVotes[i] = linearSearchArr(max_size, i+1, unsortedArr);
        tempArr[i] = totalVotes[i];
    }
    insertionSort(totalVotes, 5);
    std::cout<<std::endl;
    for(int i=0; i<5;i++){
        if(tempArr[i] == totalVotes[4]){
            std::cout<< "\nWINNER: Candidate "<< i+1<< std::endl;
        }
    }
}
```

```
void displayArr(int arr[]){
    for(int i = 0;i <max_size; i++ ){
        std::cout<< arr[i] << " ";
    }
    std::cout<<std::endl;
}
```

sortingAlgo.h:

```
template <typename T>
void insertionSort(T arr[], size_t arrSize){
    int K = 0, J, temp;
    while(K < arrSize){
        temp = arr[K];
        J = K-1;

        while(J >= 0 && temp <= arr[J]){
            arr[J+1] = arr[J];
            J--;
        }
        arr[J+1] = temp;
        K++;
    }
}
```

searchingAlgo.h:

```
template<typename T>
int linearSearchArr(int size,T item,T array[]){
    int repeating = 0;
    for (int i = 0; i < size;i++){
        if(array[i] == item){
            repeating++;
        }
        else if(array[i] > 5){
            std::cout<<"ERROR: Only numbers 1-5 are allowed";
        }
    }
    return repeating;
}
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
Sorted using Insertion Sort: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Tally of Votes: Candidate 1: Bo Dalton Capistrano: 24 Candidate 2: Cornelius Raymon Agustin: 14 Candidate 3: Deja Jayla Banaga: 17 Candidate 4: Lalla Brielle Yabut: 23 Candidate 5: Franklin Relano Castro: 22	WINNER: Candidate 1

Question: Was your developed vote counting algorithm effective? Why or why not?

- Yes it is effective because the worst case time complexity of my program is O(n^2) and the best case is O(n), it would be bad if both cases are O(n^2). I used insertion sort since its time complexity is worst case time complexity is O(n^2) and the best case is O(n) when there is a case where the array is already sorted. Unlike selection sort where both cases are O(n^2). For the searching technique I used, I used array-based linear search since it is much simpler to use and implement even though I know binary search is better when you need to sort the array. Overall my program managed to do its task properly as it was able to sort, count, and output the winning candidate.

Time Complexity of Program
Best Case: T(n) = 10n + 25
O(n)
(Happens when the array is already sorted)
Worst Case: T(n) = 2n^2 + 8n + 25
O(n^2)

CONCLUSION: In this activity, I learned about various sorting techniques, for me the easiest one to understand is

bubble sort as it is very simple in concept. Through the activities in this, I managed to know how to implement these sorting techniques using C++. The supplementary then made me use both searching and sorting techniques which made use of our previous activity. Through the supplementary also I discovered possible uses for these sorting and searching techniques such as in vote counting.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to give me clues on what to do if I'm stuck but I don't tell it to give me the code, I'm the one who thinks about how to create the code based on the clues AI gives me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.